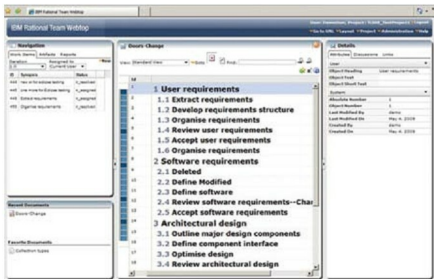


SOFTWARE TOOLS SELECTION FOR EMBEDDED SYSTEMS

Software tools greatly influence the entire embedded development cycle from designing to deployment. A proper set of software tools plays a significant role in reducing the time to design, code, debug and test the embedded systems. With the growth in hardware options, demand for more integrated, cross-platform and independent software tools is increasing



IBM's Rational Team Webtop presents information from multiple software delivery tools in a single web-based view

■ SHWETA DHADIWAL

In the embedded design and development cycle, from conceptualisation to building prototype and from coding to debugging, software tools play a very critical role. These not only ease the designing process and shorten development time but also make the system more reliable. The advent of various software tools for designing and development has reduced the time to debug and time to market. But selection of proper tools is very important to get the best results.

Embedded conceptualisation

In embedded system development, requirements define the specifications of systems or products. It is very

important to know what you want to build, what features it should have and how will it be used. However, very little care is taken to formulate the correct requirements. "Requirements are gathered from a variety of sources and encompass many aspects including user requirements, functionality, performance, compliance, safety, security, etc," expresses Kurian John, manager product delivery-Rational, India Software Lab, IBM. Talking about software tools, John adds, "There are tools that can help to manage and trace requirements at multiple levels, determine the coverage of requirements and manage changes."

Selection of components and hardware design depends on the desired functionality of the end product. With the available options, some components

may be procured from manufacturers/vendors and some may be designed in-house. Electronic design and automation (EDA) tools help to design and optimise the component-level performance for specific applications.

"Designers are now more inclined towards model-driven development," says John. System modeling helps in assigning the functionalities to hardware or software in an embedded system. Engineers use SysML (System Modeling Language) and UML (Unified Modeling Language) to create a cohesive architecture. Design and modeling software feature easy to use "drag and drop" options.

Talking about the validation process, John says, "A simulation software is used to validate the functionality to uncover defects early in the development life cycle, when it is least costly to fix." Simulators are software programs that imitate your actual program in real environment. Simulation provides a good cross-checking and helps locate the problem areas. The use of simulator significantly reduces your time-to-market and increases the reliability. Simulators also provide advantage when hardware prototypes are expensive or unavailable.

The approach of embedded design and development is somewhat different. With the advent of the concept of system-on-chip (SoC), a hardware-software co-design and co-verification technique has emerged. In this technique, the chip is adequately tested in simulation environment. There are tools developed to meet the require-